

PARADIGMAS DE LINGUAGENS DE PROGRAMAÇÃO

Prof. Rogerio Mandelli

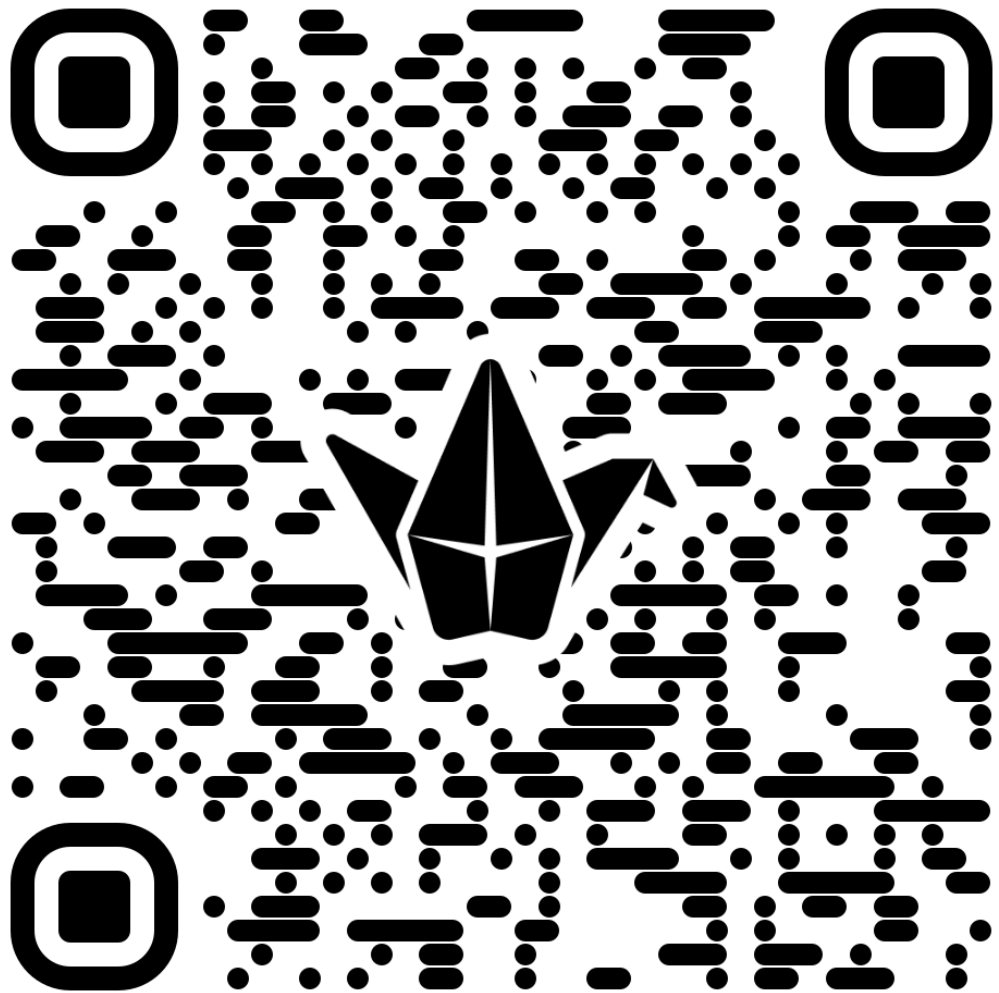
ESCOLHA VIVER
MAIS EXPERIÊNCIAS.

Aula 01 | 29 abr 2026

UVA ●
A sua melhor escolha.

Introdução





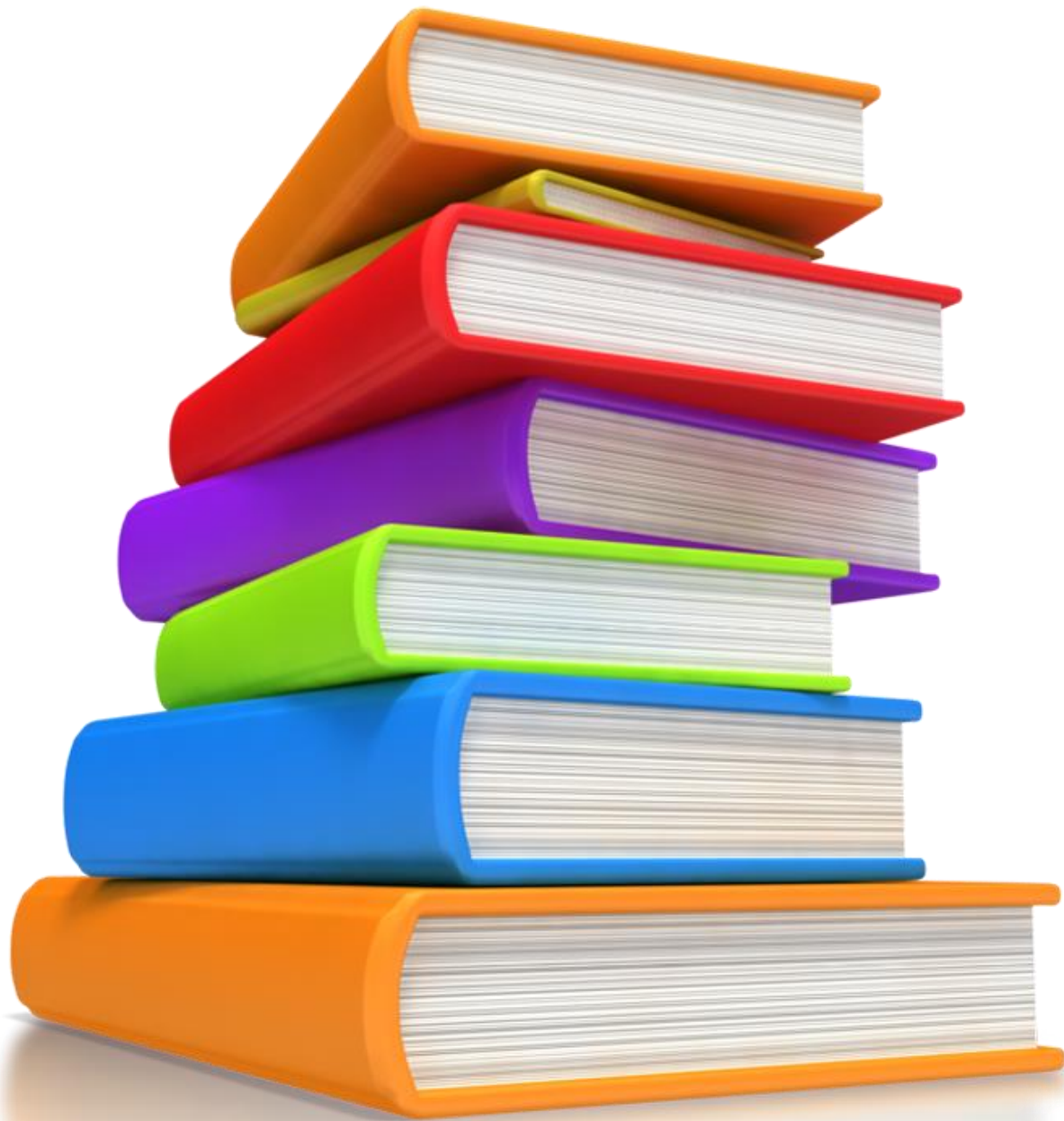
:Padlet

Workspace da Turma

Objetivo da disciplina

- **Desenvolver** a capacidade de **construir o conhecimento** associando as características, funcionamento e estrutura dos **principais paradigmas de linguagens**.





Referências Básicas

1. SEBESTA, Robert W.. **Conceitos de Linguagens de Programação**. 11ª Ed. Porto Alegre: RS, Bookman, 2018. [Minha Biblioteca]
2. TUCKER, Allen B., NOONAN, Robert E.. **Linguagens de programação: princípios e paradigmas**. 2ª Ed. Porto Alegre: RS, AMGH, 2010. [Minha Biblioteca]
3. MELO, Ana Cristina Vieira. **Princípios de Linguagens de Programação**, Ed. Edgard Blücher Ltda, 2014. [Minha Biblioteca]

Avaliação

✓ Entrega Avaliativa
(100%)

x ADI (0,0 a 1,0)

- Ex.: Nota EA
(10,0) x ADI
(0,75) = 7,5

Média mínima = 6,0



UVA

Ciclo Básico das Engenharias
Disciplina: Projeto I - Turma A Noite
Professor: Rogerio Mandelli

		Fase 1							
Ordem	NOME	1	2	3	4	5	6		TOTAL GRUPO
1	Cintia Fausto		3,0	3,0	2,0	2,0	0,0		10,0
2	Daniel David Damiani	2,0		4,5	1,5	2,0	0,0		10,0
3	Kariche Moraes Silva Veiga (GP)	2,5	3,5		2,5	1,5	0,0		10,0
4	Luan de Britto Pereira	2,5	3,0	3,0		1,8	0,0		10,3
5	Pedro Henrique Farias de Almeida	2,0	3,0	3,0	2,0		0,0		10,0
6	Vinícius Nobre Ribeiro	0,0	0,0	0,0	0,0	0,0			0,0
MÉDIA ALUNO		1,8	2,5	2,7	1,6	1,5	0,0		2,7
		0,7	0,9	1,0	0,6	0,5	0,0		



Objetivos

- Compreender o conceito e o propósito das linguagens de programação.
- Distinguir sintaxe, semântica e pragmática.
- Conhecer a classificação e as propriedades desejáveis das linguagens.
- Entender os conceitos de abstração de dados e de processo.
- Executar e comparar o programa “Hello World” em C, Python e Java.
- Exercícios.

Por que estudar linguagens de programação?

1. Compreender fundamentos de qualquer linguagem moderna.
2. Escolher a linguagem adequada para cada problema.
3. Desenvolver pensamento crítico e comparativo entre paradigmas.
4. Aumentar a autonomia na aprendizagem de novas linguagens.





Por que estudar linguagens de programação?

1. Compreender fundamentos de qualquer linguagem moderna

- Toda linguagem, seja C, Python, Java, etc. compartilha **conceitos fundamentais**:
 - variáveis, tipos de dados, estruturas de controle, funções, escopo, abstração, modularização.
- Ao entender esses fundamentos: não se “aprende uma linguagem”, mas sim **aprende a aprender linguagens**.
- Isso cria **transferência de conhecimento**:
 - quem domina C entende como a memória funciona;
 - quem entende Python compreende alto nível e tipagem dinâmica;
 - quem entende Java entende orientação a objetos e encapsulamento.



Por que estudar linguagens de programação?

2. Escolher a linguagem adequada para cada problema

- Nem toda linguagem é boa para tudo:
 - C → sistemas embarcados, baixo nível, desempenho.
 - Python → análise de dados, IA, scripts rápidos.
 - Java → aplicações corporativas, multiplataforma.
- Profissional deve **avaliar critérios** como desempenho, segurança, manutenção, comunidade, e compatibilidade.
- Essa escolha é parte do **pensamento de engenharia de software**: decidir com base em requisitos, não em preferência pessoal.

The background of the slide is a complex, abstract graphic. It features a network of white lines connecting various points, creating a web-like structure. Overlaid on this are several geometric shapes, including triangles and polygons, in shades of blue and purple. There are also circular nodes at some of the intersection points. In the lower-left and upper-left areas, there are faint, stylized representations of binary code (0s and 1s) and a line graph with axes and data points. The overall color palette is dominated by deep blues, purples, and whites.

Por que estudar linguagens de programação?

3. Desenvolver pensamento crítico e comparativo entre paradigmas

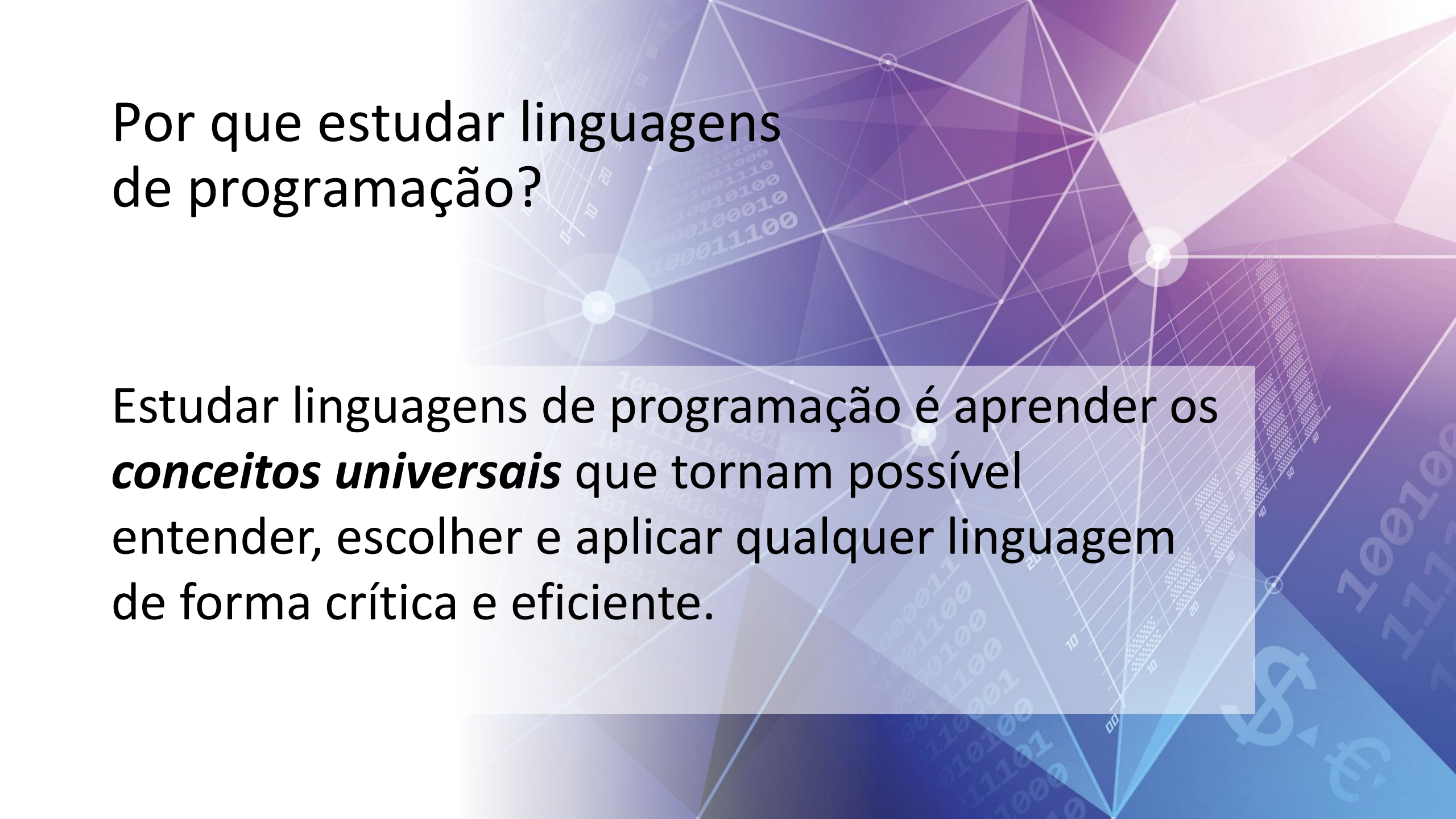
- Cada paradigma (imperativo, funcional, lógico, orientado a objetos, concorrente) **oferece uma visão diferente do mesmo problema.**
- Estudar paradigmas amplia o repertório mental e estimula o **pensamento crítico**: “como esse problema seria resolvido de outro modo?”.
- Essa comparação: leva à **maturidade conceitual**: compreender não só *como programar*, mas *por que programar dessa forma*.



Por que estudar linguagens de programação?

4. Aumentar a autonomia na aprendizagem de novas linguagens

- **Mercado de tecnologia:** dinâmico; surgem novas linguagens e frameworks constantemente (ex.: Kotlin, Rust, Go, Julia).
- **Objetivo da disciplina: formar programadores adaptáveis**, capazes de entender rapidamente uma nova linguagem porque dominam seus **conceitos de base**.
- Busca-se: deixar de depender de tutoriais e passar a **raciocinar sobre o funcionamento da linguagem**.



Por que estudar linguagens
de programação?

Estudar linguagens de programação é aprender os ***conceitos universais*** que tornam possível entender, escolher e aplicar qualquer linguagem de forma crítica e eficiente.

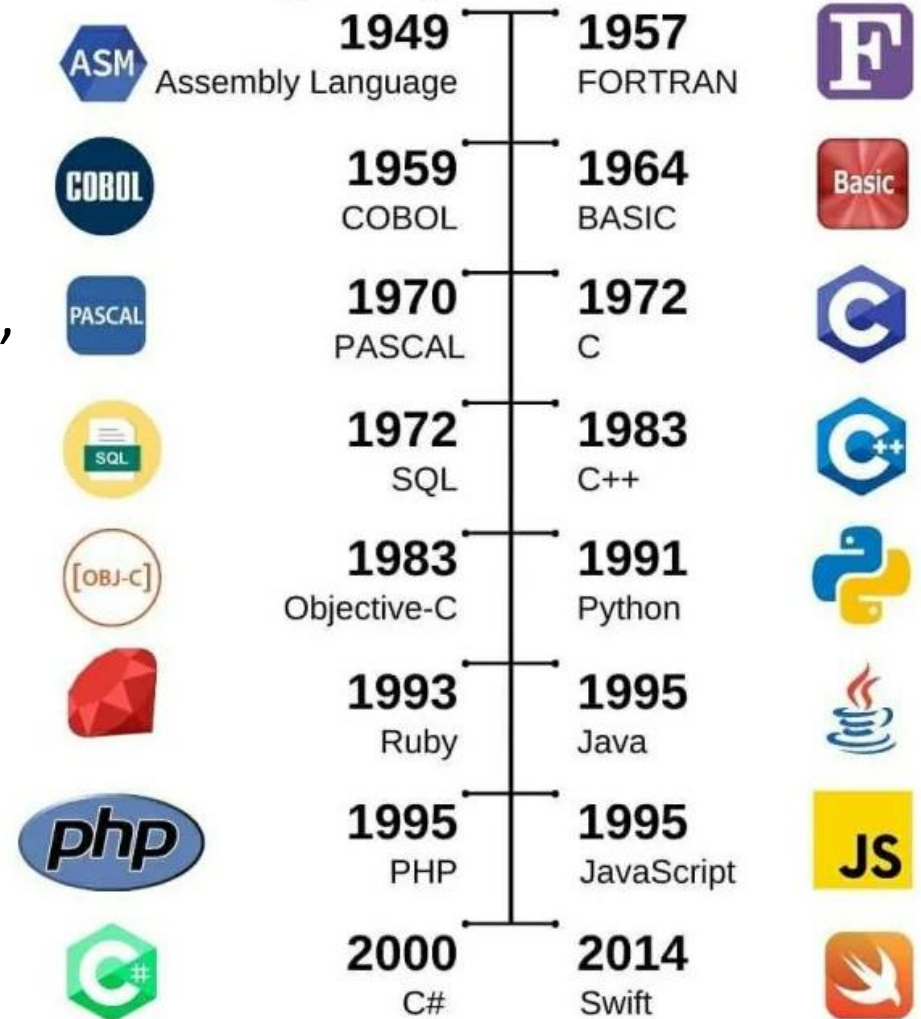
Evolução das Linguagens (Genealogia)

Evolução das Linguagens (Genealogia)

- C → base para sistemas e influência em C++, Java, C#
- Lisp → origem do paradigma funcional
- Prolog → paradigma lógico
- Python → linguagem multiparadigma moderna
- Java → OO + portabilidade (JVM)

Linguagens não surgem do nada — são evolução de ideias anteriores.

History of Popular Programming Languages: A timeline





Linha evolutiva simplificada

- Máquina → Assembly → C
- C → C++ → Java
- Lisp → Haskell
- Prolog → IA simbólica
- Python → integração de paradigmas

O que é uma Linguagem de Programação?

- Uma **linguagem de programação**:
 - **Sistema formal de comunicação** entre humanos e computadores.
 - Composto por **símbolos, regras sintáticas e semânticas**.
 - Isto permite **especificar algoritmos e manipular dados** para resolver problemas computacionais.
 - **Ponte entre o raciocínio humano e a execução automática pela máquina.**



O que é uma Linguagem de Programação?

- **Conjunto de símbolos, regras e convenções**
 - **Símbolos:** palavras-chave, operadores, identificadores, pontuação e números usados para expressar instruções.
Exemplo: `if`, `while`, `{}`, `;`, `+`, `==`, etc.
 - **Regras:** determinam **como combinar os símbolos** de forma válida (gramática da linguagem).
Exemplo: um comando `if` precisa obrigatoriamente de uma condição e um bloco de execução.
 - **Convenções:** padrões culturais e de estilo, como indentação, nomeação de variáveis e boas práticas (Python, por exemplo, é sensível à indentação).





```
</div>  
</div>
```

```
<div class="ui-box product-package">  
  <div class="ui-box-title">  
    <div class="ui-box-body">  
      <ul class="product-package-list">  
        <li class="package-item">  
          <span class="package-price">
```

Onde entram os paradigmas?

- Imperativo → como fazer
- Declarativo → o que fazer
- OO → organização em objetos
- Funcional → funções puras
- Lógico → regras e inferência

Paradigmas = formas diferentes de pensar soluções



```
</div>  
</div>
```

```
<div class="ui-box product-package">  
<div class="ui-box-title">  
<div class="ui-box-body">
```

```
<ul class="product-package-list">  
<li class="package-item">  
  <span class="package-item">
```

```
</li>  
</ul>  
</div>
```

Componentes Essenciais de uma LP

- 1) **Sintaxe:** forma e estrutura do código.
- 2) **Semântica:** significado das instruções.
- 3) **Pragmática:** uso e aplicação prática.



```
</div>  
</div>
```

```
<div class="ui-box product-package">  
<div class="ui-box-title">  
<div class="ui-box-body">
```

```
<ul class="product-package-list">  
<li class="package-item">  
  <span class="package-item">
```

```
</li>  
</ul>  
</div>
```

Componentes Essenciais de uma LP

- 1) **Sintaxe** → “como se escreve”
 - Define a **estrutura formal** das instruções.
 - É equivalente à **gramática de uma língua natural**.
 - Exemplos a seguir em:

 Python

 C

 Java

*Todos expressam a mesma ideia, mas têm **sintaxes diferentes**.*



```
</div>  
</div>
```

```
<div class="ui-box product-package">  
<div class="ui-box-title">  
<div class="ui-box-body">
```

```
<ul class="product-package-list">  
<li class="package-item">  
  <span class="package-item">
```

```
</li>  
</ul>  
</div>
```

Componentes Essenciais de uma LP



```
if x > 0:  
    print("positivo")
```

- Sintaxe simples e legível.
- Uso de **dois pontos (:)** e **indentação obrigatória** para definir o bloco.
- **Sem ponto e vírgula.**
- **Sem declaração de tipo explícita** (tipagem dinâmica).



```
</div>  
</div>
```

```
<div class="ui-box product-package">  
  <div class="ui-box-title">  
    <div class="ui-box-body">  
      <ul class="product-package-list">  
        <li class="package-item">  
          <span class="package-item">
```

Componentes Essenciais de uma LP



C

```
if (x > 0) {  
    printf("positivo");  
}
```

- Estrutura formal e delimitada por **parênteses e chaves**.
- **Uso obrigatório de ponto e vírgula.**
- Exige **inclusão de biblioteca** (`#include <stdio.h>`) e **declaração de variável** (`int x;`).
- **Tipagem estática e explícita.**



```
</div>  
</div>
```

```
<div class="ui-box product-package">  
  <div class="ui-box-title">  
    <div class="ui-box-body">
```

```
      <ul class="product-package-list">  
        <li class="package-item">  
          <span class="package-item">
```

```
        </li>  
      </ul>  
    </div>  
  </div>
```

```
</div>
```

Componentes Essenciais de uma LP

☕ Java

```
public static void main(String[] args)  
{  
    int x = 5;  
    if (x > 0)  
    {  
        System.out.println("positivo");  
    }  
}
```

- Estrutura semelhante à de C, mas **orientada a objetos**.
- O comando `System.out.println()` pertence à **classe System** do pacote `java.lang`.
- Precisa estar dentro de um **método**, como acima.



```
</div>  
</div>
```

```
<div class="ui-box product-package">  
<div class="ui-box-title">  
<div class="ui-box-body">
```

```
<ul class="product-package-list">  
<li class="package-item">  
  <span class="package-price">
```

```
</li>  
</ul>  
</div>
```

Componentes Essenciais de uma LP

2. Semântica → “o que significa”

- Dá **sentido ao código** escrito — o que realmente acontece quando o programa é executado.
- Exemplo:
 - $x = x + 1$ → em C ou Python, significa *incrementar o valor de x*.
 - Se for $x == 1$, o significado muda completamente (comparação).
- Linguagens diferentes podem ter a **mesma sintaxe, mas semânticas distintas** (ex.: = em SQL é comparação, não atribuição).



```
</div>  
</div>
```

```
<div class="ui-box product-package">  
  <div class="ui-box-title">  
    <div class="ui-box-body">
```

```
      <ul class="product-package">  
        <li class="package">  
          <span class="package">
```

```
        </li>  
      </ul>  
    </div>  
  </div>
```

Componentes Essenciais de uma LP

2. Semântica → “o que significa”

- Compreender o elo entre **código** e **comportamento lógico**, isto é, o *significado real* de cada instrução.
- Lógica: **fundamento da semântica**.
 - Toda linguagem de programação: tem um conjunto de **regras lógicas internas** que determinam como as expressões são avaliadas e como o fluxo se comporta.
- Exemplos:
 - `if` (condição) → traduz-se em uma **avaliação lógica booleana** (verdadeiro/falso).
 - Operadores como `&&`, `||`, `!` seguem as **leis da lógica proposicional**.



```
</div>  
</div>
```

```
<div class="ui-box product-package">  
  <div class="ui-box-title">  
    <div class="ui-box-body">  
      <ul class="product-package">  
        <li class="package-item">  
          <span class="package-item">
```

Componentes Essenciais de uma LP

3. Pragmática → “quando e por que usar”

- **Sintaxe** trata da *forma* e a **semântica** do *significado*, a **pragmática** é o *uso efetivo* — e isso envolve **fatores humanos, sociais e contextuais** que vão além da teoria da linguagem.



```
</div>  
</div>
```

```
<div class="ui-box product-package">  
<div class="ui-box-title">  
<div class="ui-box-body">
```

```
<ul class="product-package-list">  
<li class="package-item">  
  <span class="package-price">
```

```
</li>  
</ul>  
</div>
```

Componentes Essenciais de uma LP

3. Pragmática → “quando e por que usar”

- Pragmática responde à pergunta:

“Qual linguagem (ou abordagem) é mais adequada para este problema, neste ambiente, com esta equipe?”

- Isso inclui:
 - **Contexto técnico:** desempenho, interoperabilidade, hardware disponível.
 - **Contexto humano:** facilidade de aprendizado, legibilidade, curva de adoção.
 - **Contexto social:** comunidade ativa, suporte, ferramentas e frameworks.
 - **Contexto temporal:** longevidade e manutenção de longo prazo.
- Exemplo prático:
 - Um sistema embarcado em microcontrolador pode exigir C, enquanto uma aplicação web moderna se beneficia de JavaScript.



```
</div>  
</div>
```

```
<div class="ui-box product-package">  
<div class="ui-box-title">  
<div class="ui-box-body">
```

```
<ul class="product-package-list">  
<li class="package-item">  
  <span class="package-item">
```

```
</li>  
</ul>  
</div>
```

Componentes Essenciais de uma LP

3. Pragmática → “quando e por que usar”

- Refere-se ao **uso da linguagem em contextos reais**: eficiência, legibilidade, manutenção, compatibilidade.
- Envolve decisões de projeto:
 - Escolher Python pela **facilidade de leitura**.
 - Escolher C por **performance e controle de hardware**.
 - Escolher Java por **robustez e portabilidade**.
- Dimensão **humana e prática** da linguagem de programação.

Abstração

- **Abstração:**
 - grau de proximidade entre a linguagem e o pensamento humano (ou a máquina).
 - quanto **maior a abstração, mais fácil de programar, mas menor o controle** sobre o hardware.



Abstração

- Capacidade de **enxergar o essencial** de um problema, **sem se preocupar com os detalhes** de como ele é feito.
- Na programação: significa **simplificar a complexidade**:
 - Esconder o que não é importante no momento e mostrar apenas o que é necessário para resolver a tarefa.
- **Exemplo:**
 - Quando você dirige um carro, não precisa saber como funciona o motor — basta usar o volante, os pedais e o câmbio.



Abstração de Dados e de Processo

- **Abstração de dados:**
representação de informações em estruturas lógicas.
- **Abstração de processo:**
organização de operações em funções reutilizáveis.



Abstração de Dados

- Forma de **representar informações** de modo organizado e lógico, sem se preocupar com como elas são armazenadas na memória.
 - Em vez de pensar em `bits` e `bytes`, usamos **estruturas de dados** como listas, vetores, registros, objetos, etc.
 - Programador foca no **significado** das informações, não na forma física delas.
- **Exemplo:**
 - Em vez de pensar em “endereço de memória”, representamos um aluno assim:

Python

```
aluno = {"nome": "Ana", "idade": 20,  
"curso": "Computação"}
```



Abstração de Dados

- Forma de **organizar ações e operações** em partes menores e reutilizáveis — chamadas **funções** ou **métodos**.
 - Ideia: criar **blocos lógicos de código** que executam uma tarefa e podem ser usados várias vezes.

Python

```
def calcularArea(raio):  
    return 3.14 * raio ** 2
```



Abstração de Dados-Processo

Python

```
raio = 5  
area = calcularArea(raio)  
print("Área do círculo:", area)
```

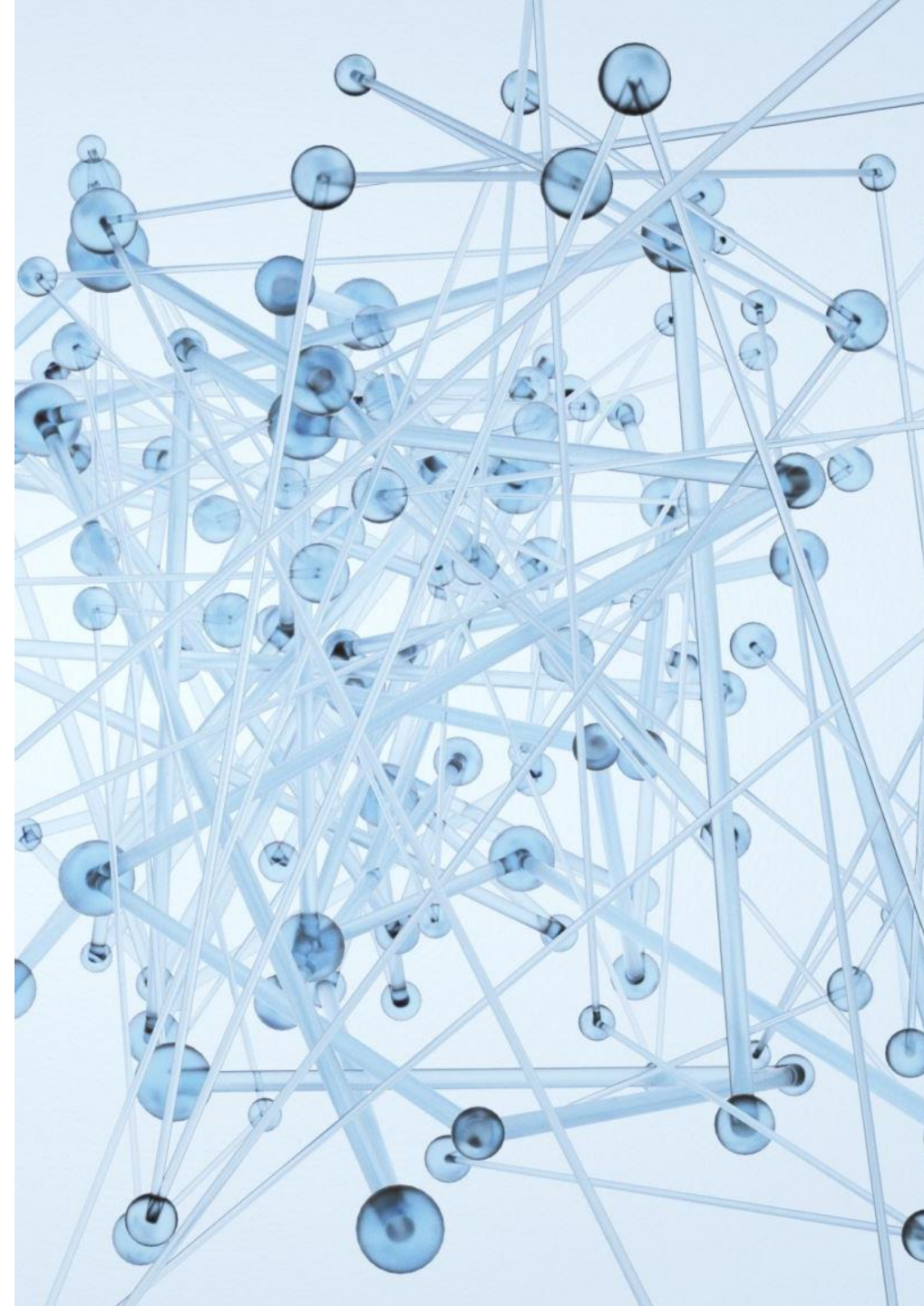
- Trabalha-se em um nível **conceitual** — não precisa saber *como o número é armazenado* nem *como a multiplicação é implementada* internamente.



Linguagens e Níveis de Abstração

1) Linguagem de Máquina (nível 0 – mais perto do hardware)

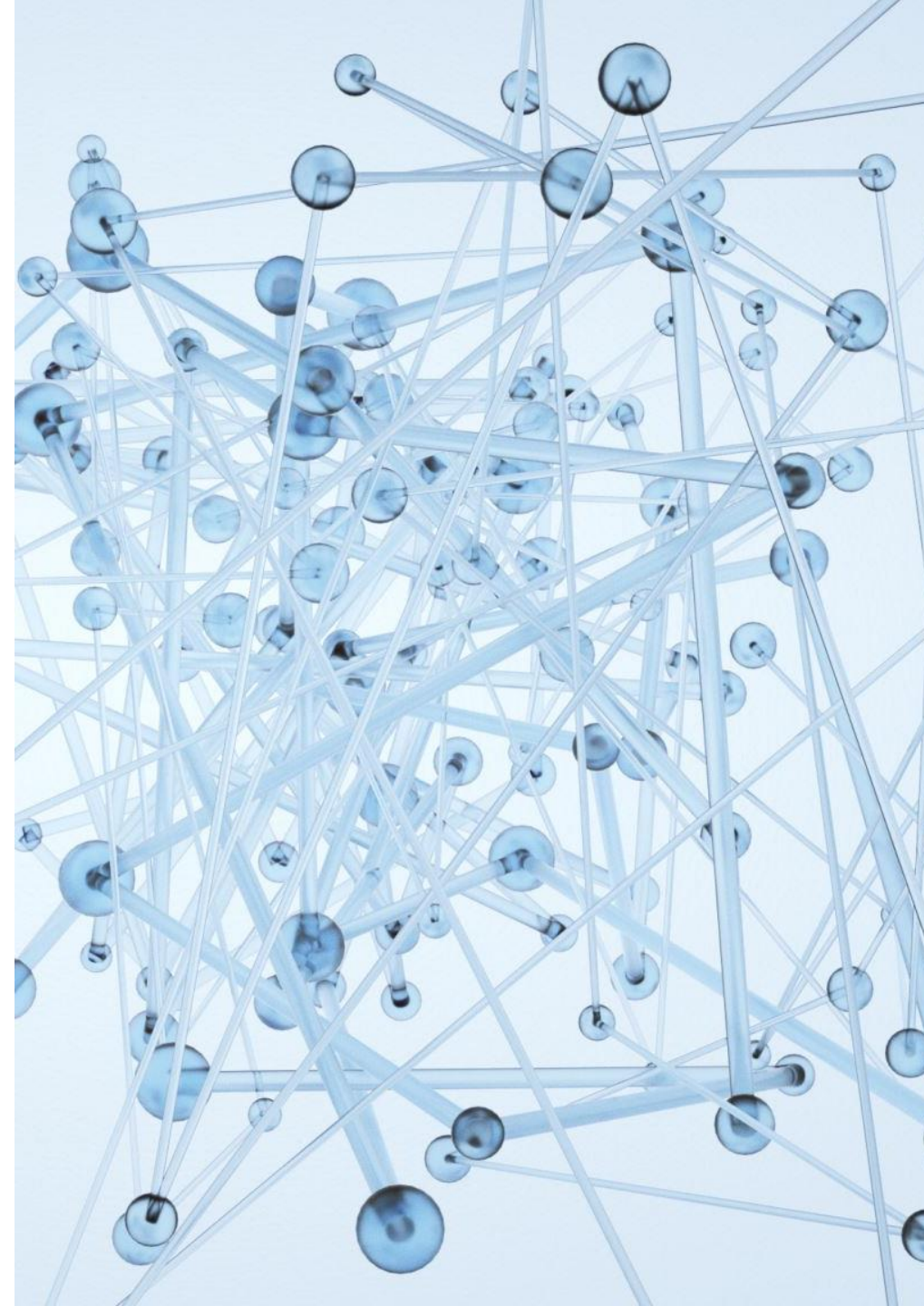
- **O que é:** sequência de **números binários (0 e 1)** que o **processador entende diretamente**.
 - Cada sequência representa uma **instrução específica** (ex.: somar, mover, comparar).
- **Características:**
 -  Máximo desempenho e controle.
 -  Pouca portabilidade (cada tipo de processador usa um conjunto diferente de códigos).
 -  Muito difícil de ler e programar manualmente.
- **Exemplo (x86, em hexadecimal):**
 - B8 01 00 00 00 → significa “**mova o número 1 para o registrador EAX**”
- **Uso típico:**
 -  Código gerado automaticamente pelo **compilador**.
 -  Utilizado em **drivers, firmwares e sistemas operacionais**.



Linguagens e Níveis de Abstração

2) Assembly (baixo nível)

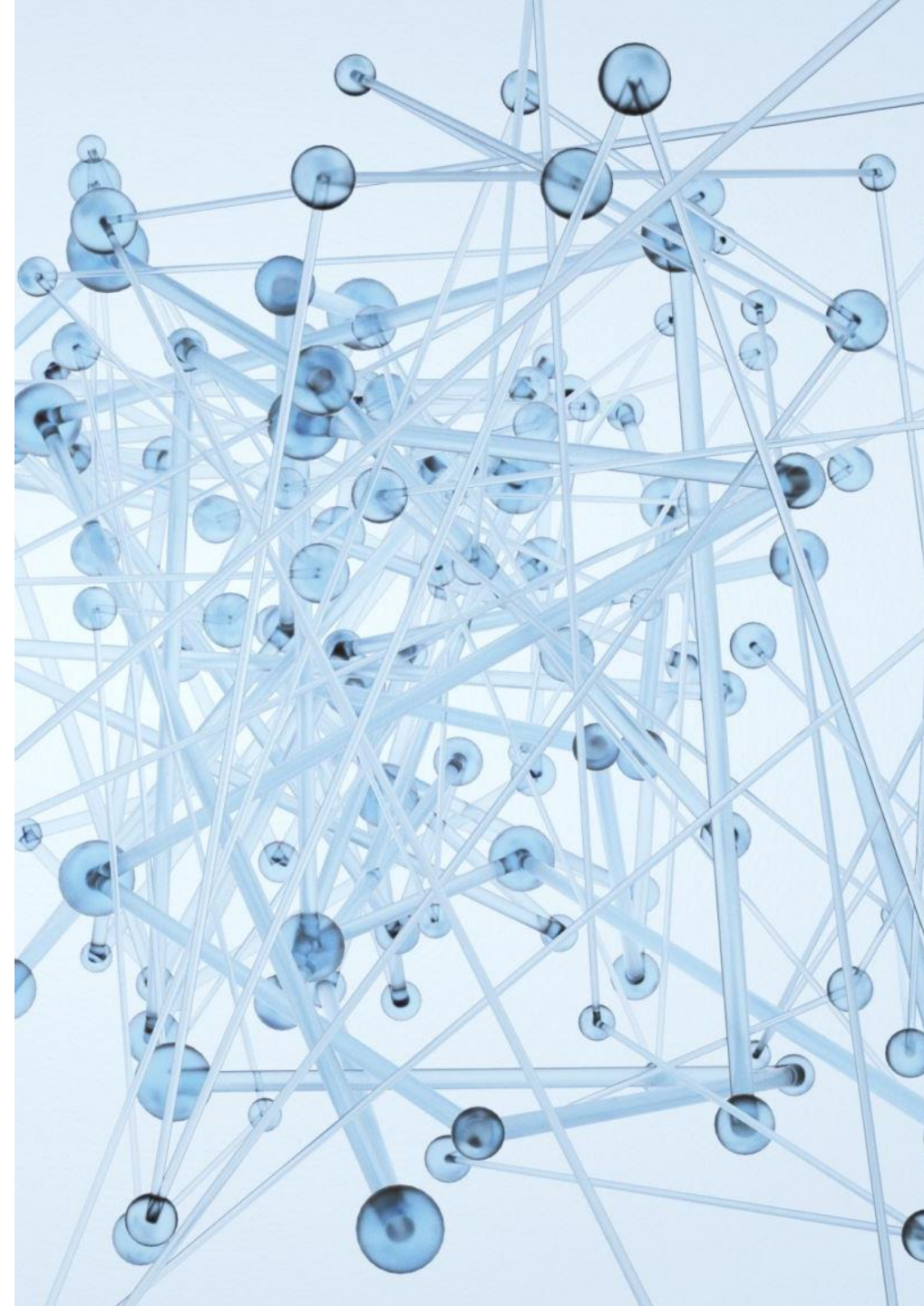
- **O que é:** linguagem que usa **palavras curtas (mnemônicos)** para representar comandos de máquina.
 - É como escrever **“MOV AX, 1”** em vez de **“B8 01 00 00 00”**.
 - Cada instrução ainda **corresponde diretamente a uma operação do processador**.
- **Características:**
 - 🌀 Ainda **específica de cada tipo de processador** (x86, ARM, etc.).
 - 🎯 Permite controlar **registradores e endereços de memória** diretamente.
 - ⚡ Muito **rápida e eficiente**, mas **difícil de manter** e pouco portátil.
- **Exemplo (x86):**
 - `MOV AX, 1`
 - `MOV BX, AX`
 - `ADD AX, BX`
 - → Move valores entre registradores e soma.
- **Uso típico:**
 - 💎 **Bootloaders** (inicialização do sistema)
 - 💎 **Rotinas críticas** de desempenho
 - 💎 **Código gerado automaticamente** por compiladores em partes otimizadas.



Linguagens e Níveis de Abstração

3) Alto Nível (linguagens gerais)

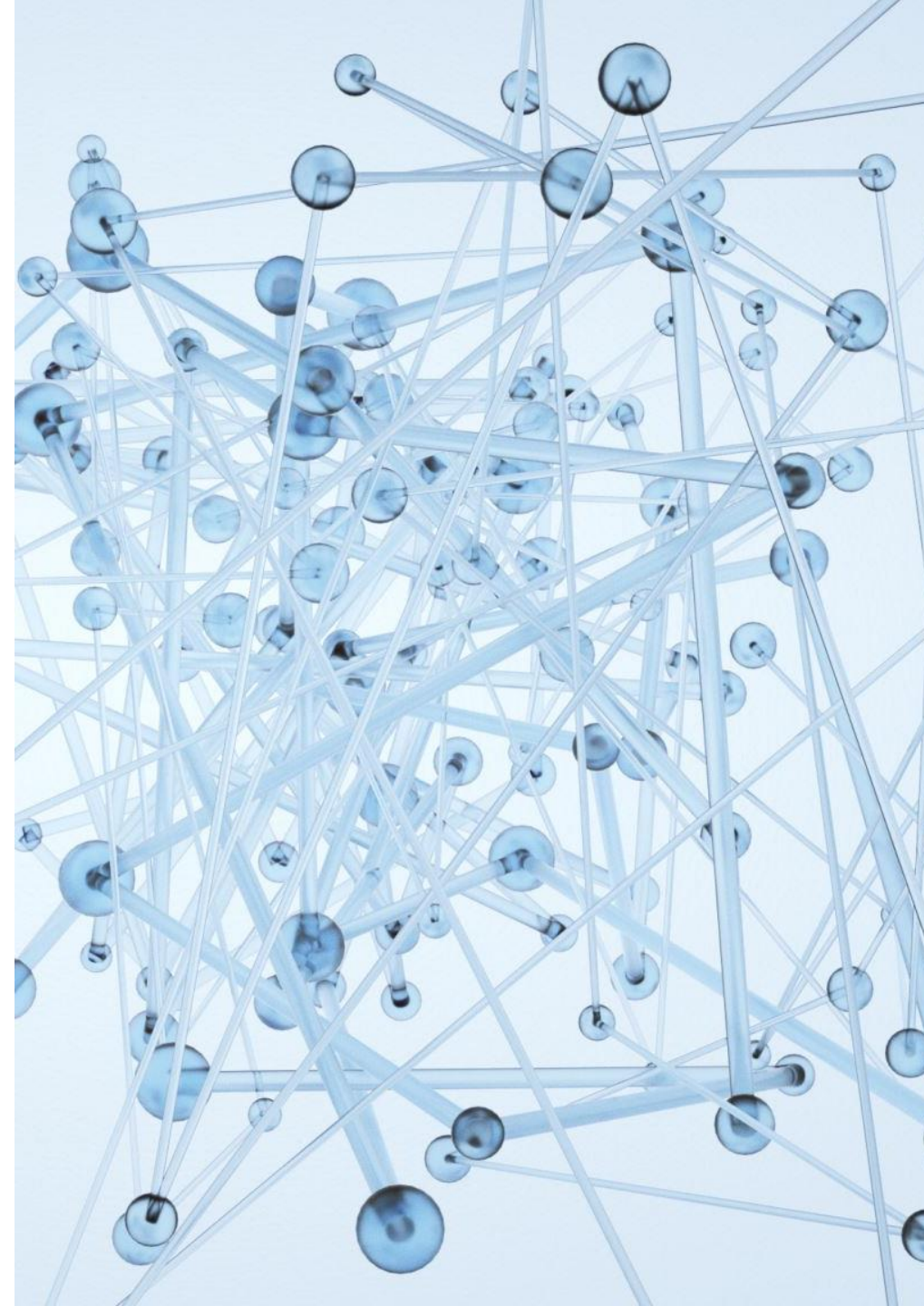
- **O que são:** Linguagens que descrevem **algoritmos com sintaxe próxima da linguagem humana**.
 - → **Compilador ou interpretador** traduz essas instruções para o código de máquina.
 - → Programador **não precisa lidar com registradores ou endereços de memória**.
- **Exemplos:**
 - C, C++, Java, Python, Go, C#, JavaScript, Rust
- **Características principais:**
 -  **Mais legíveis e portáteis** (rodam em diferentes sistemas).
 -  **O compilador/interpretador faz o trabalho pesado** (tradução, otimização, execução).
 -  **Maior produtividade**, com menor risco de erros de hardware.
- **Diferentes comportamentos:**
 - **C:** mais próximo do hardware (controle e desempenho).
 - **Java/C#:** geram **bytecode**, executado em uma máquina virtual (JVM/CLR).
 - **Python/JavaScript:** interpretadas, focadas em **facilidade e rapidez de desenvolvimento**.



Linguagens e Níveis de Abstração

4) Muito Alto Nível / 4GL / Declarativas / DSLs

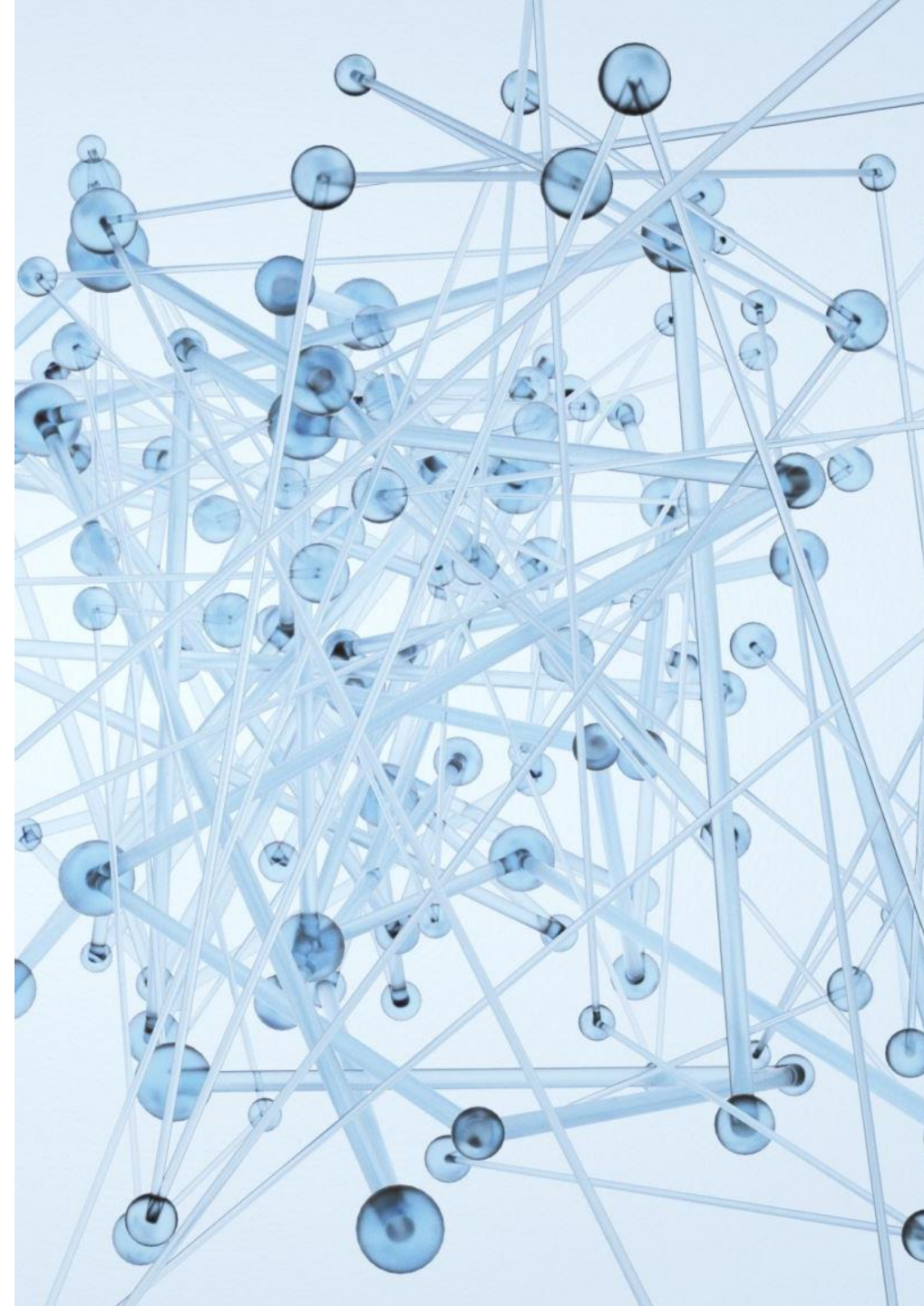
- **O que são:**
 - Linguagens em que o programador **descreve o que deseja obter**, e **não precisa especificar passo a passo como fazer**.
 - O sistema interpreta e decide **como executar** as instruções.
- **Exemplos:**
 - 🧩 **SQL** – consultas a bancos de dados
 - 📊 **R / Matlab** – computação científica e estatística
 - 🌐 **HTML / CSS** – estrutura e estilo de páginas
 - 🤖 **Prolog** – programação lógica
 - 📄 **Haskell** – paradigma funcional
 - 🔧 **Regex / Gradle / DSLs** – linguagens específicas de domínio (*Domain-Specific Languages*)
- **Vantagens:**
 - ✅ **Alta produtividade** em tarefas específicas
 - ✅ **Foco no resultado**, não no processo
 - ✅ **Menos código, menos erros**
- **Desvantagens:**
 - ⚙️ **Menor controle** sobre os detalhes de execução.
 - 🔒 Dependência de **mecanismos internos** da linguagem ou ferramenta.



Linguagens e Níveis de Abstração

4) Muito Alto Nível / 4GL / Declarativas / DSLs

- **O que são:**
 - Linguagens em que o programador **descreve o que deseja obter**, e **não precisa especificar passo a passo como fazer**.
 - O sistema interpreta e decide **como executar** as instruções.
- **Exemplos:**
 - 🧩 **SQL** – consultas a bancos de dados
 - 📊 **R / Matlab** – computação científica e estatística
 - 🌐 **HTML / CSS** – estrutura e estilo de páginas
 - 🤖 **Prolog** – programação lógica
 - 📄 **Haskell** – paradigma funcional
 - 🔧 **Regex / Gradle / DSLs** – linguagens específicas de domínio (*Domain-Specific Languages*)
- **Vantagens:**
 - ✅ **Alta produtividade** em tarefas específicas
 - ✅ **Foco no resultado**, não no processo
 - ✅ **Menos código, menos erros**
- **Desvantagens:**
 - ⚙️ **Menor controle** sobre os detalhes de execução.
 - 🔒 Dependência de **mecanismos internos** da linguagem ou ferramenta.





Propriedades Desejáveis

1. Legibilidade
2. Escrita simples
3. Confiabilidade
4. Eficiência
5. Portabilidade
6. Manutenibilidade



Propriedades Desejáveis

1. Legibilidade

- Refere-se à facilidade com que um código pode ser lido e compreendido por outras pessoas.
- Linguagem legível favorece a colaboração, a revisão e a correção de erros.
- Em Python, a legibilidade é alta:



Propriedades Desejáveis

1. Legibilidade

- O mesmo trecho em C exige mais detalhes:

```
for (int i = 0; i < tamanho; i++)  
{  
    printf("%s\n", lista[i]);  
}
```

- Em Java, é mais estruturado, mas também mais verboso:



Propriedades Desejáveis

2. Escrita simples

- Uma linguagem simples evita complexidades desnecessárias, redundâncias e exceções às regras.
 - **Python:** busca simplicidade, escondendo detalhes de memória e tipos.
 - **C:** o programador precisa declarar tudo manualmente e cuidar de ponteiros — o que é mais complexo, mas oferece maior controle.
 - **Java:** fica no meio: exige declarações explícitas, mas automatiza muita coisa (como o gerenciamento de memória).



Propriedades Desejáveis

3. Confiabilidade

- Uma linguagem confiável reduz a chance de erros e comportamentos imprevisíveis.
 - **Python:** confiável na execução, mas depende fortemente de testes, pois é interpretado e tipado dinamicamente.
 - **C:** programador tem liberdade total, mas também pode causar falhas de memória se não for cuidadoso.
 - **Java:** bom exemplo: o sistema de tipos forte e o tratamento de exceções (try / catch) tornam o programa mais seguro.



Propriedades Desejáveis

4. Eficiência

- Está ligada ao desempenho: tempo de execução e uso de memória.
 - **Python:** menos eficiente em velocidade, mas compensa com rapidez de desenvolvimento e clareza.
 - **C:** extremamente eficiente por compilar diretamente para código de máquina e permitir acesso ao hardware.
 - **Java:** eficiente o bastante, mas a JVM adiciona uma camada intermediária.



Propriedades Desejáveis

5. Portabilidade

- Uma linguagem portátil permite que o mesmo código rode em diferentes sistemas.
 - **Python:** também é bastante portátil, bastando ter o interpretador instalado.
 - **C:** por outro lado, exige recompilação em cada sistema e pode depender de detalhes do hardware.
 - **Java:** destaca-se com sua máquina virtual (JVM), permitindo “escreva uma vez, execute em qualquer lugar”.



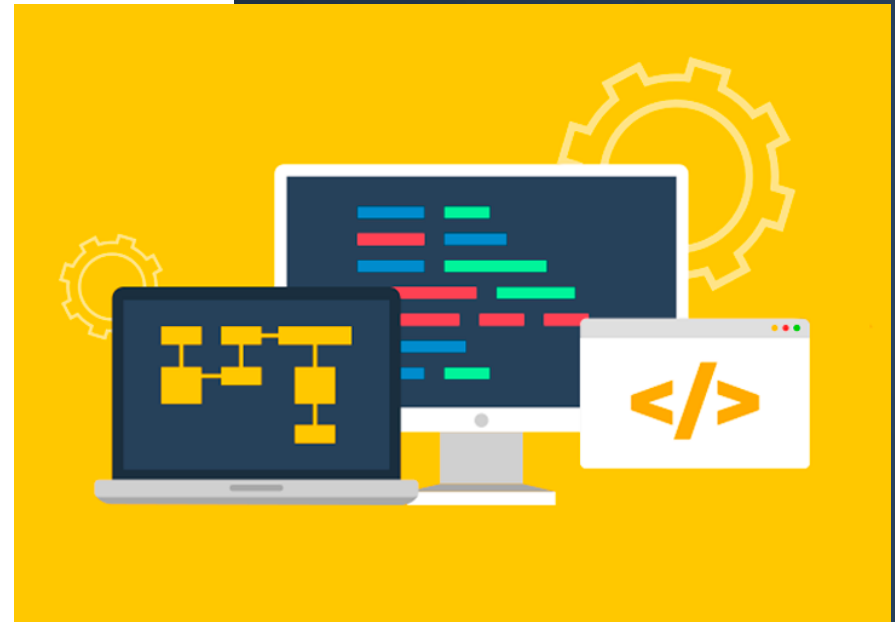
Propriedades Desejáveis

6. Manutenibilidade

- Facilidade de atualizar, corrigir e expandir o código ao longo do tempo. Códigos legíveis e modulares favorecem a manutenção.
 - **Python:** legibilidade alta e módulos simples facilitam refatorações rápidas; porém o tipado dinâmico pode esconder erros.
 - **Java:** forte apoio à manutenção em sistemas grandes graças a tipagem estática, encapsulamento, interfaces/abstrações e um ecossistema robusto de ferramentas. Estruturas tornam mudanças grandes mais seguras e previsíveis, ainda que o código seja mais verboso.
 - **C:** oferece controle fino, mas exige gestão manual de memória e dependências o que aumenta o risco de regressões e torna refatorações mais trabalhosas.

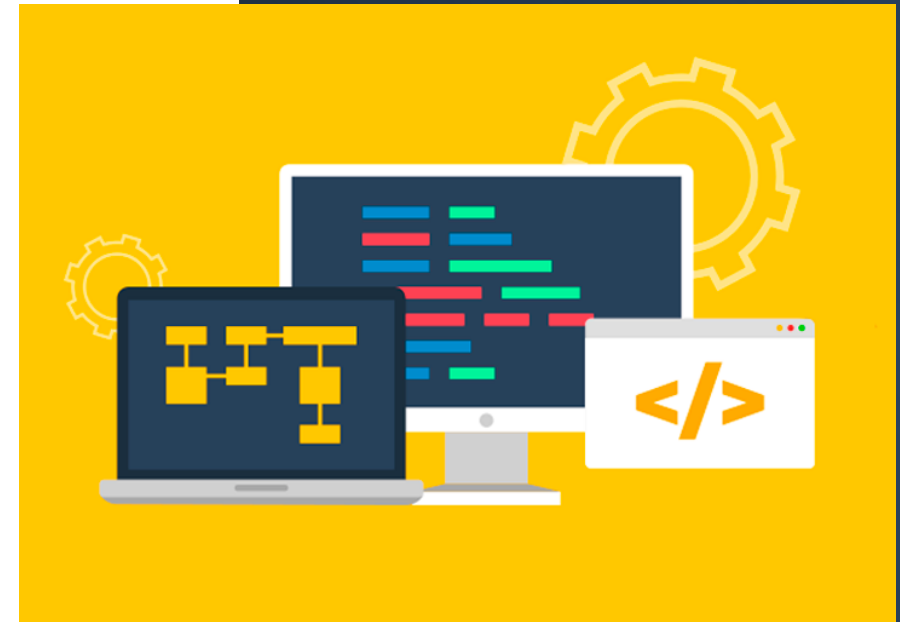
Ambientes de Desenvolvimento

- **C — VS Code + GCC — Compilada**
 - C: linguagem **compilada**, ou seja, o código fonte precisa ser traduzido **antes da execução** para linguagem de máquina (arquivo binário).
 - Tradução é feita por um **compilador**, sendo o **GCC (GNU Compiler Collection)** o mais comum.
 - **VS Code**: usado apenas como **editor**, enquanto o **GCC** faz o trabalho de **compilar e gerar o executável**.
 - Isso garante **alta eficiência e velocidade de execução**, mas qualquer erro de sintaxe impede a geração do programa.



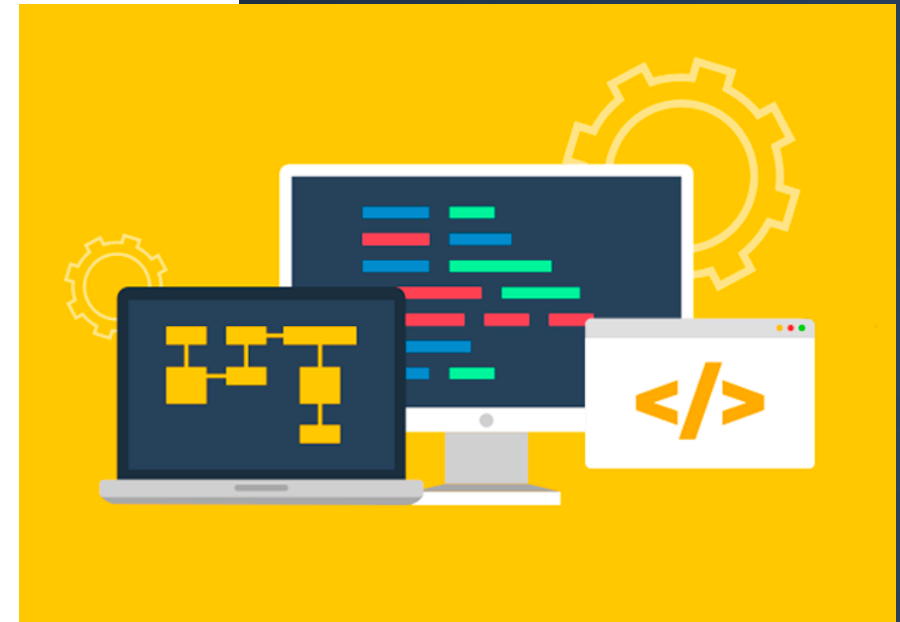
Ambientes de Desenvolvimento

- **Python — VS Code / IDLE — Interpretada**
 - Linguagem **interpretada**, ou seja, o código é **executado linha a linha**, diretamente por um **interpretador**.
 - Não há necessidade de compilar antecipadamente; basta salvar o arquivo .py e executá-lo.
 - Isso torna o desenvolvimento **mais rápido e interativo**, ideal para ensino e prototipagem.
 - Pode-se usar o **VS Code** (com extensão Python) ou o ambiente próprio **IDLE**, que já vem com o instalador da linguagem.



Ambientes de Desenvolvimento

- **Java — IntelliJ / VS Code — Compilada + JVM**
 - Java é uma linguagem **compilada e interpretada ao mesmo tempo**.
 - Código fonte (.java) é **compilado** pelo **javac** para um formato intermediário chamado **bytecode** (.class).
 - Bytecode **não é executado** diretamente **pelo sistema operacional**, mas sim pela **JVM (Java Virtual Machine)**, que o **interpreta ou JIT-compila** em tempo de execução.
 - Isso garante **portabilidade**: o mesmo bytecode roda em qualquer computador que tenha a JVM instalada.
 - Ambientes comuns: **IntelliJ IDEA**, **Eclipse**, ou **VS Code** com extensão Java.



1º Programa: Hello World em C

```
#include <stdio.h>
int main()
{
    printf("Hello, World!");
    return 0;
}
```

- Compilado via GCC e executado pelo sistema.



1º Programa: Hello World em Python

```
print("Hello, World!")
```

- Interpretado pelo interpretador Python.



1º Programa: Hello World em Java

```
class HelloWorld
{
    public static void main(String[] args)
    {
        System.out.println("Hello, World!");
    }
}
```

- Compilado em bytecode e executado pela JVM.



Comparando as Linguagens

C: 4 linhas – compilada –
função principal obrigatória

Python: 1 linha –
interpretada – sintaxe direta

Java: 5 linhas – compilada –
estrutura orientada a
objetos

Exercício 1.1

Comparação inicial entre Python, C e Java

Desenvolva um programa simples que leia o nome de uma pessoa e um número inteiro. Em seguida, o programa deve informar se o número digitado é positivo, negativo ou zero.

Depois de executar o programa em **Python, C e Java**, compare:

- 1.Qual linguagem exigiu menos linhas?
- 2.Qual linguagem exigiu mais estrutura obrigatória?
- 3.Em qual delas a tipagem ficou mais explícita?
- 4.O que muda na forma de entrada e saída de dados?
- 5.O que isso revela sobre o nível de abstração de cada linguagem?

